ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 222E

COMPUTER ORGANIZATION
PROJECT 1 REPORT

CRN : 23727

LECTURER : Do.. Dr. Gökhan İnce

GROUP MEMBERS:

150230089 : Hüseyin Teke

150230040 : Aysun Dilara Doğan

SPRING 2026

Contents

1 INTRODUCTION **1**

2 MATERIALS AND METHODS **2**

2.1 16 Bit Register Design and Implementation 2
2.2 Register File Design and Implementation 3
2.3 Address Register File Design and Implementation 4
2.4 Data Register Design and Implementation 5
2.5 Data Memory Unit (DMU) Design and Implementation 6
2.6 Instruction Register Design and Implementation 7
2.7 Instruction Memory Unit (IMU) Design and Implementation 8
2.8 Arithmetic Logic Unit (ALU) Design and Implementation 10
2.9 System Integration and Data Routing 11

3 RESULTS **13**

3.1 Module-Level Simulation Results 13
3.2 Integrated ALU System Performance 15

4 DISCUSSION **17**

4.1 Combinatorial vs. Sequential Logic Trade-offs 18
4.2 Handling Signed Arithmetic and Overflow 18
4.3 Modular Integration and Bus Routing 18

5 CONCLUSION **19**

REFERENCES **20**

1 INTRODUCTION

The primary objective of this project is the modular design and implementation of a comprehensive 16-bit processor datapath and an Arithmetic Logic Unit (ALU) system using Verilog Hardware Description Language (HDL). The system is architected to perform a wide range of arithmetic, logical, and memory-related operations through a hierarchical approach consisting of four main developmental stages.The first phase of the design involves the construction of a versatile 16-bit register capable of fundamental operations such as clearing, loading, incrementing, and decrementing based on a 2-bit control signal ($FunSel$) and an enable input. These basic units are then integrated into more complex structures: a Register File (RF) containing eight general-purpose and scratch registers (R1-R4, S1-S4), and an Address Register File (ARF) which manages critical system registers including the Program Counter (PC), Stack Pointer (SP), and Address Register (AR).Furthermore, the project covers the implementation of specialized memory management units. This includes a Data Memory Unit (DMU) integrated with Random Access Memory (RAM) for dynamic data handling, and an Instruction Memory Unit (IMU) linked with Read-Only Memory (ROM) for instruction retrieval and registration.The core of the system is a 16-bit Arithmetic Logic Unit (ALU), which is designed to execute sixteen distinct operations, ranging from basic two's complement arithmetic to complex circular, logical, and arithmetic shifts. A dedicated Flag Register is implemented to store the state of the Zero (Z), Carry (C), Negative (N), and Overflow (O) flags, providing essential feedback for the system's operational status. Finally, the entire organization is interconnected using a series of multiplexers to form a synchronized system operating under a single global clock, ensuring efficient data routing between processing, storage, and memory units.

2 MATERIALS AND METHODS

2.1 16 Bit Register Design and Implementation

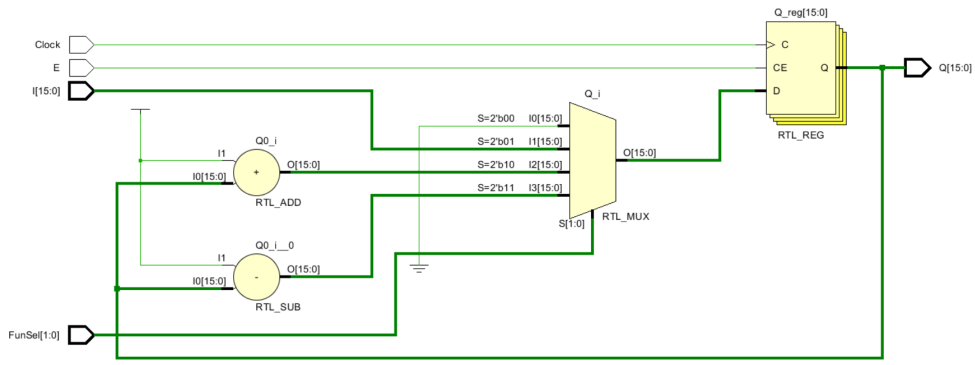

Figure 1: Elaborated RTL Schematic of the 16 bit register module

The 16-bit register is the fundamental synchronous storage element used throughout the processor architecture. As shown in the elaborated RTL schematic in Figure 1, the module is designed to handle four specific operations governed by a 2-bit control signal ($FunSel$) and a master enable bit (E).The implementation details are as follows:

- **Functionality:** The register supports four critical operations: Clear (00), Load (01), Increment (10), and Decrement (11).
- **Control Logic:** When the Enable (E) input is high, the register performs the operation selected by $FunSel$ on the positive edge of the clock. If E is low, the register retains its current value (Q), ensuring data stability.
- **Modular Design:** This register is implemented as a standalone module to be reused across the Register File (RF) and Address Register File (ARF) units.

Source Code: The complete Verilog implementation of this module can be accessed at the following link:
Click here to view Register16bit.v on GitHub

- Functionality: The register supports four operations: Clear (00), Load (01), Increment (10), and Decrement (11).
- Control Logic: When the Enable (E) input is high (1), the register performs the operation selected by $FunSel$ on the positive edge of the clock. If E is low (0), the register enters a "Retain Value" state, ignoring the $FunSel$ inputs and preserving its current state (Q).

- Arithmetic Operations: The increment and decrement functions ($Q+1$ and $Q-1$) allow the register to act as a counter, which is a critical feature for modules like the Program Counter (PC) and Stack Pointer (SP) in later parts of the project.

- Implementation Strategy: The module is designed using a behavioral always @(posedge Clock) block to ensure that all transitions are synchronized with the system clock, preventing asynchronous glitches during data transfers.

2.2 Register File Design and Implementation

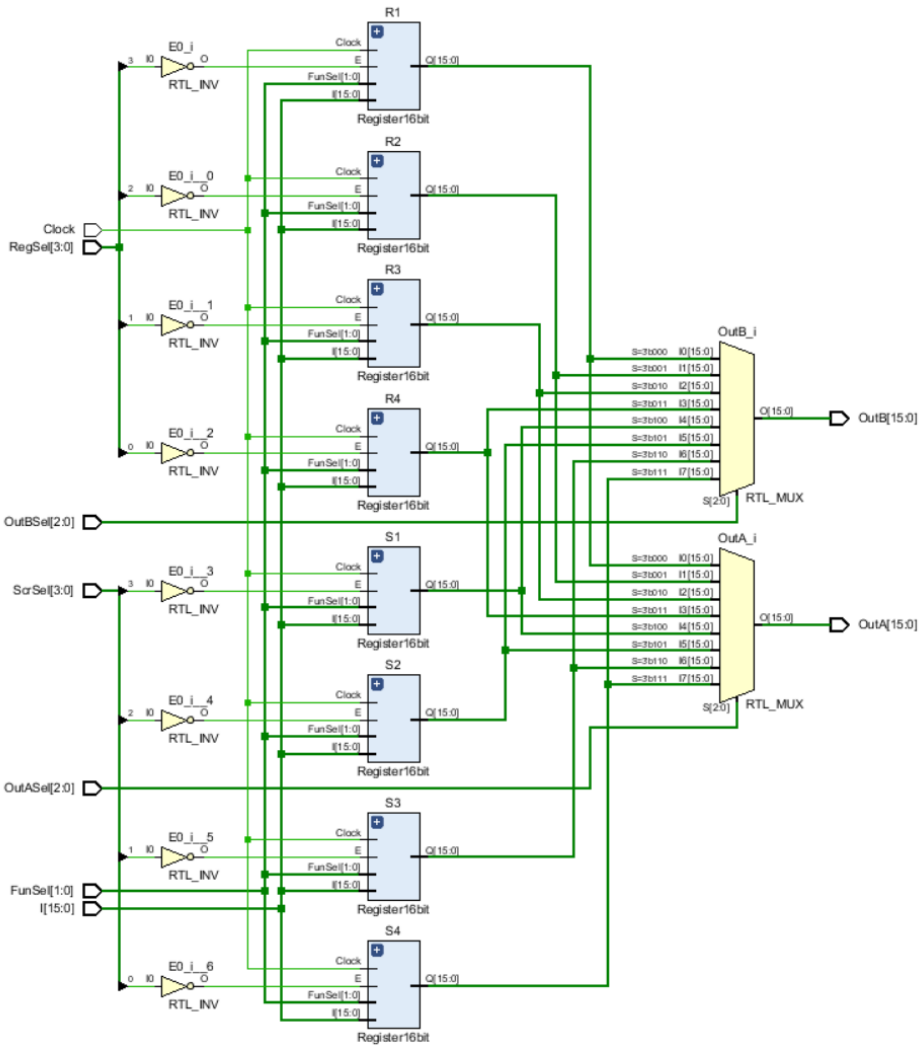

Figure 2: Elaborated RTL Schematic of the Register File Module

The Register File (RF) is a hierarchical structure designed to provide high-speed data storage and retrieval for the processor. As shown in the elaborated RTL schematic in Figure 2, the RF module integrates eight instances of the previously designed 16-bit register. The implementation details and architectural choices are as follows:

- **Internal Organization:** The RF consists of four general-purpose registers (R1, R2, R3, R4) and four scratch registers (S1, S2, S3, S4). Each register is a 16-bit synchronous unit capable of independent operations.
- **Output Selection (Mux Logic):** Two 8-to-1 multiplexers are utilized to route data to the $OutA$ and $OutB$ buses. The $OutASel$ and $OutBSel$ 3-bit control signals determine which of the eight registers is exposed to the ALU at any given time.
- **Function Application and Enable Logic:** The 2-bit $FunSel$ signal determines the operation (Clear, Load, Increment, Decrement) to be applied to the registers. Crucially, the 4-bit $RegSel$ and $ScrSel$ signals act as advanced enable controllers, allowing the system to apply a function to multiple registers simultaneously or retain their values individually.
- **Clock Synchronization:** To maintain data integrity and prevent race conditions within the datapath, all register updates within the RF are synchronized to the positive edge of the global system clock.

Source Code: The complete Verilog implementation of the Register File module can be accessed at the following link:
Click here to view RegisterFile.v on GitHub

2.3 Address Register File Design and Implementation

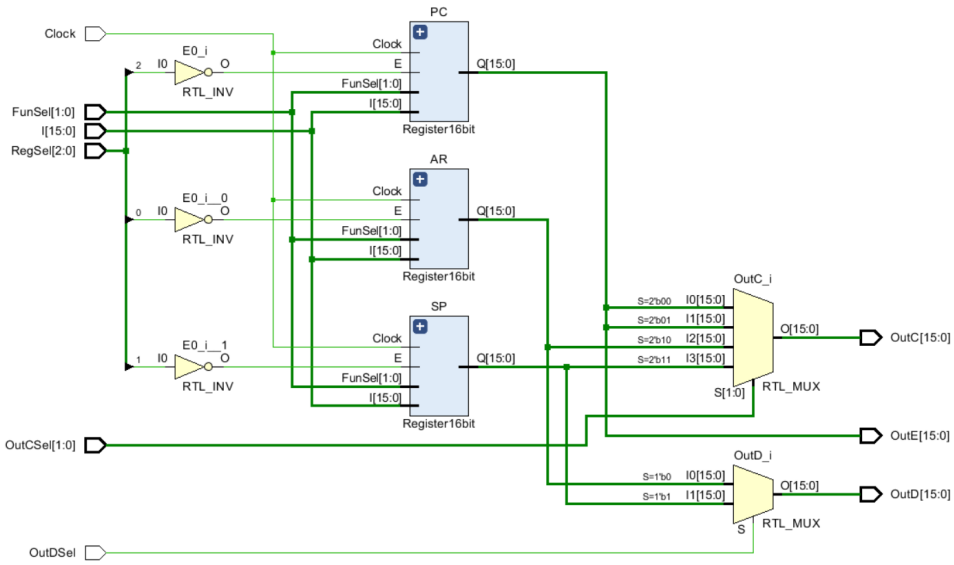

Figure 3: Elaborated RTL Schematic of the ARF Module

The Address Register File (ARF) is a specialized storage unit dedicated to managing memory addresses and program flow control. Unlike the general-purpose Register File,

the ARF contains registers with specific architectural roles. As illustrated in the RTL schematic in Figure 3, the implementation details are as follows:

- **Register Composition:** The ARF consists of three 16-bit registers: the Program Counter (PC), the Stack Pointer (SP), and the Address Register (AR).
- **Output Routing:** The module features three output buses:
  - *OutC* and *OutD*: These are controlled by *OutCSel* and *OutDSel* multiplexer signals to provide addresses to the memory units.
  - *OutE*: This bus is hardwired to the Program Counter (PC) to ensure the instruction fetch logic always has access to the current instruction pointer.
- **Function Selection:** Similar to other register units, the ARF utilizes a 2-bit *FunSel* signal to perform Clear, Load, Increment, and Decrement operations. This is particularly critical for the PC, which requires frequent increments during sequential execution.
- **Selective Enable Logic:** A 3-bit *RegSel* signal is implemented to enable specific combinations of PC, SP, and AR for simultaneous updates or individual state retention.

Source Code: The complete Verilog implementation of the Address Register File module can be accessed at the following link:
Click here to view AddressRegisterFile.v on GitHub

2.4 Data Register Design and Implementation

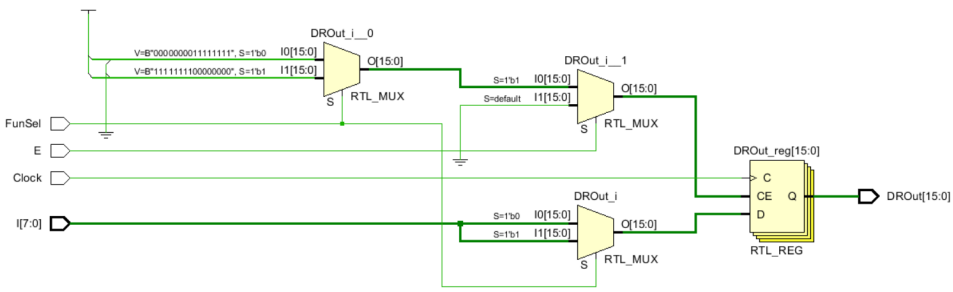

Figure 4: Elaborated RTL Schematic of the Data Register Module

The Data Register (DR) is a specialized 16-bit storage unit designed to interface with 8-bit memory buses. As illustrated in the RTL schematic in Figure 4, the implementation features and operational logic are as follows:

- **16-bit Storage with 8-bit Interface:** Although the register stores a full 16-bit binary value, its input bus is limited to 8 bits to match the memory data width.
- **Two-Cycle Loading Logic:** To fully load a 16-bit word, the register operates in two distinct stages controlled by the $FunSel$ signal:
  - **Load LSB ($FunSel = 0$):** The 8-bit input is loaded into the lower half (bits 7-0) of the register.
  - **Load MSB ($FunSel = 1$):** The 8-bit input is loaded into the upper half (bits 15-8) of the register.
- **Synchronous Operation:** All loading operations are gated by an Enable (E) signal and occur on the positive edge of the clock. When $E = 0$, the register retains its current 16-bit value.
- **System Role:** The DR serves as the primary data handler within the Data Memory Unit (DMU), facilitating the transfer of multi-byte data between the RAM and the internal registers.

Source Code: The complete Verilog implementation of the Data Register module can be accessed at the following link: *Click here to view DataRegister.v on GitHub*

2.5 Data Memory Unit (DMU) Design and Implementation

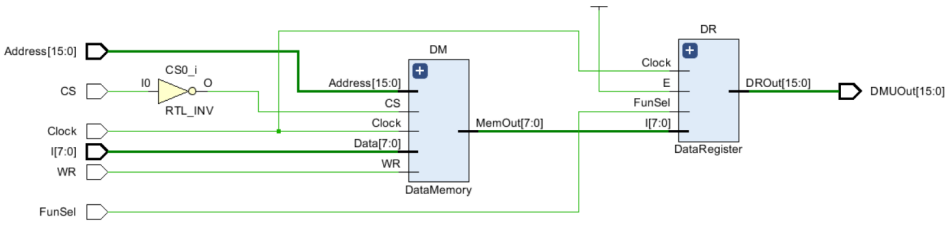

Figure 5: Elaborated RTL Schematic of the Data Memory Unit (DMU) Module

The Data Memory Unit (DMU) is a sophisticated subsystem that integrates Random Access Memory (RAM) with a specialized Data Register (DR) to facilitate efficient data exchange within the processor. As illustrated in the elaborated RTL schematic in Figure 5, the unit is designed to bridge the gap between the 16-bit internal architecture and the 8-bit memory interface.The implementation and operational details are as follows:

- **RAM Integration:** The unit incorporates a provided RAM module that utilizes a 16-bit address and an 8-bit data bus. The RAM operation is controlled by a Write

Enable (WR) signal and a Chip Select (CS) signal, where $CS = 0$ enables the memory.

- **Data Handling with DR:** To manage 16-bit data transfers, the DMU uses the previously designed Data Register (DR). Since the RAM's data input is limited to 8 bits, the DR acts as a buffer that loads or stores data in two separate clock cycles (LSB and MSB).

- **Control Signals:** The DMU is governed by three primary inputs: CS, WR, and $FunSel$. The WR input of the DMU is directly connected to the RAM's WR input to determine read or write states, while the $FunSel$ signal controls the byte-loading functionality of the internal DR.

- **Addressable Memory Space:** The 16-bit address input, typically sourced from the Address Register File (ARF), allows the DMU to access a significant memory range while maintaining synchronized timing with the global system clock.

Source Code: The complete Verilog implementation of the Data Memory Unit can be accessed at the following link: *Click here to view DataMemoryUnit.v on GitHub*
Source Code: The complete Verilog implementation of the Data Memory can be accessed at the following link: *Click here to view DataMemory.v on GitHub*

2.6 Instruction Register Design and Implementation

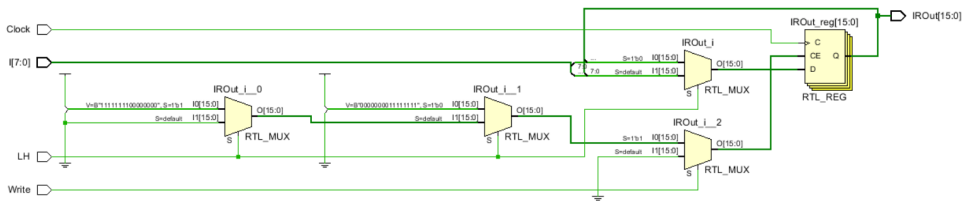

Figure 6: Elaborated RTL Schematic of the Instruction Register Module

The Instruction Register (IR) is a critical 16-bit storage unit designed to hold the current operation code fetched from the program memory. Similar to the Data Register, it operates with an 8-bit input bus, necessitating a multi-cycle load process.The architectural and operational features are as follows:

- 16-bit Capacity with 8-bit Loading: The IR stores a complete 16-bit instruction, but due to the 8-bit input constraint, it loads the data in two halves.

- L'/H Control Logic: The loading process is governed by the L'/H (Lower/Higher) signal. When $Write = 1$, a 0 on this signal loads the 8-bit input into the lower byte (bits 7-0), while a 1 loads it into the upper byte (bits 15-8).

- Synchronous State Retention: When the $Write$ signal is 0, the register ignores the input and retains its current 16-bit value, ensuring the instruction remains stable during execution.

- Output Interfacing: The register provides a 16-bit output ($IROut$) which is essential for the system's control unit to decode and execute the fetched instruction.

Source Code: The complete Verilog implementation of the Instruction Register module can be accessed at the following link: *Click here to view DataMemoryUnit.v on GitHub*

2.7 Instruction Memory Unit (IMU) Design and Implementation

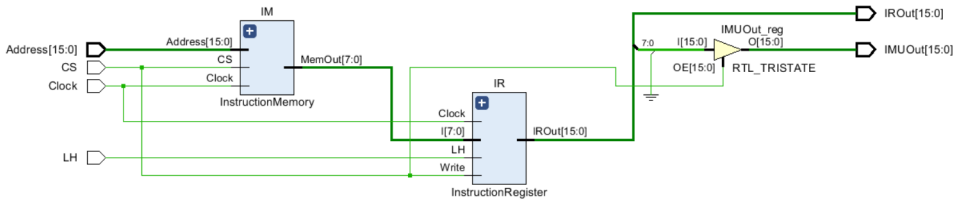

Figure 7: Elaborated RTL Schematic of the Instruction Memory Unit (IMU) Module

The Instruction Memory Unit (IMU) is the primary subsystem responsible for storing the program code and fetching instructions to be executed by the processor. It integrates a Read-Only Memory (ROM) with a 16-bit Instruction Register (IR). The core functionality of reading data from ROM and loading it into the IR is detailed below:

- **ROM Interfacing and Instruction Fetching:**

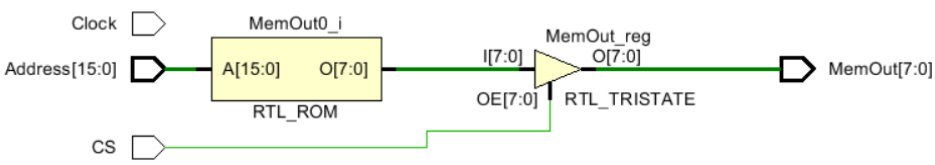

Figure 8: Elaborated RTL Schematic of the Instruction Memory Module

The IMU utilizes a ROM module where the program's instructions are stored. Data is fetched from the ROM based on a 16-bit *Address* input, which is typically pro-

vided by the Program Counter (PC). This operation conducted via Instruction Memory module.

- **Control and Enable Logic:** The read operation is strictly controlled by a Chip Select (CS) signal. The ROM and IR are only activated when $CS = 1$; otherwise, the IR retains its previous instruction state.

- **Instruction Output Interface:** Once fully loaded, the IMU provides two distinct outputs for the rest of the system:

  - **IROut:** Provides the complete 16-bit instruction directly from the IR for decoding.
  - **IMUOut:** Provides a 16-bit value containing only the lower 8 bits of the instruction, with the upper 8 bits padded with zeros.

Source Code: The complete Verilog implementation of the Instruction Memory Unit can be accessed at the following link: *Click here to view InstructionMemoryUnit.v on GitHub*

Source Code: The complete Verilog implementation of the Instruction Memory can be accessed at the following link: *Click here to view InstructionMemory.v on GitHub*

2.8 Arithmetic Logic Unit (ALU) Design and Implementation

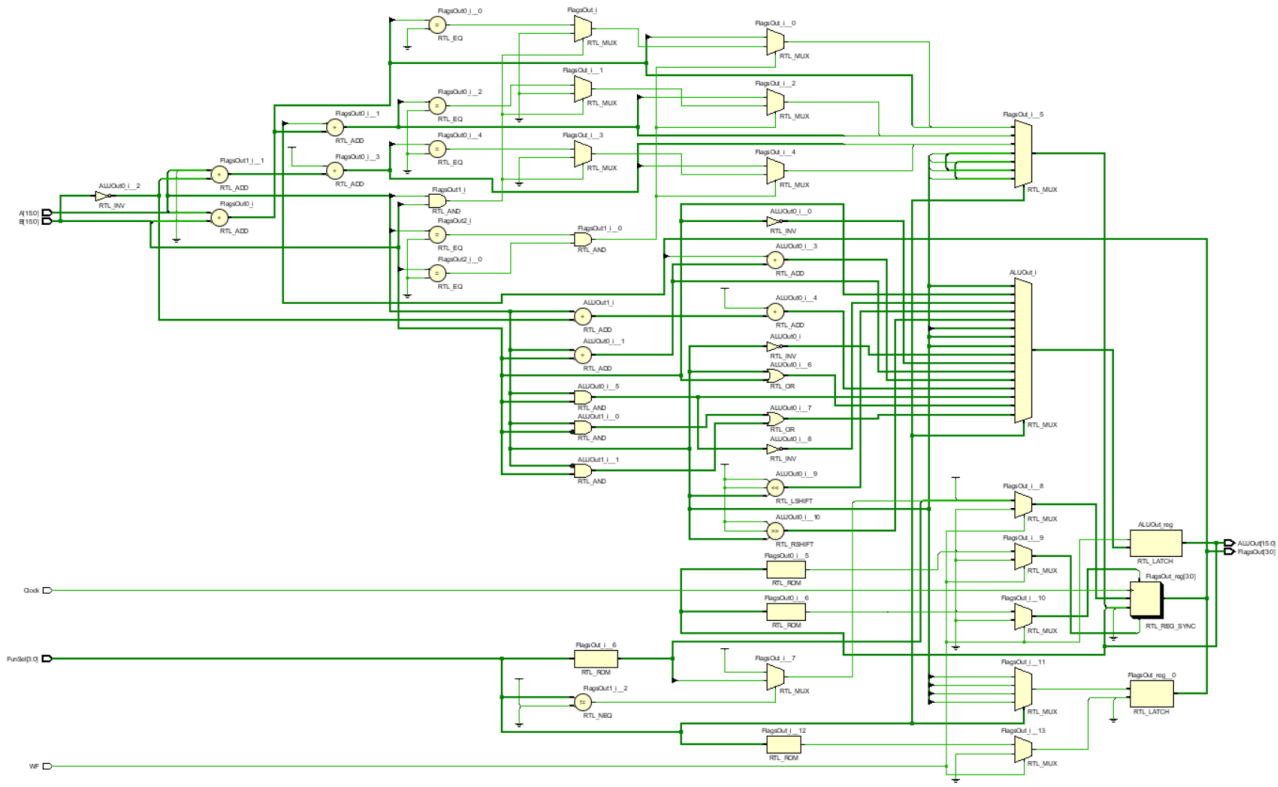

Figure 9: Elaborated RTL Schematic of the ALU and Flag Registers

The Arithmetic Logic Unit (ALU) is the primary computational core of the system, designed to process two 16-bit inputs (A and B) and produce a 16-bit result ($ALUOut$). The design follows a modular approach where arithmetic, logical, and shift operations are multiplexed based on a 4-bit $FunSel$ signal.The implementation details and operational logic are as follows:

- **Functionality:** The ALU supports 16 distinct operations, including 2's complement arithmetic (Addition, Subtraction), logical operations (AND, OR, XOR, NAND), and five types of shift operations (LSL, LSR, ASR, CSL, CSR).

- **Combinational ALU Output:** The $ALUOut$ signal is implemented using combinational logic. This ensures that the result of the operation selected by $FunSel$ is calculated and available on the bus immediately after the operands are stable, without waiting for a clock edge.

- **Sequential Flag Management (Z, C, N, O):** Unlike the main output, the status flags are stored in a dedicated 4-bit Flag Register. These flags are updated sequentially on the positive edge of the clock only when the Write Flag (WF) signal is

enabled. This sequential approach prevents race conditions and ensures that the flags remain stable for the next instruction cycle.

- **Zero (Z):** Set if the *ALUOut* is exactly zero.
- **Carry (C):** Set if an arithmetic operation generates a carry-out or if a shift operation ejects a '1'.
- **Negative (N):** Set if the result is negative (MSB is 1).
- **Overflow (O):** Set if a signed arithmetic operation results in a value outside the representable 16-bit range.

Source Code: The complete Verilog implementation of the ALU can be accessed at the following link: *Click here to view ArithmeticLogicUnit.v on GitHub*

2.9 System Integration and Data Routing

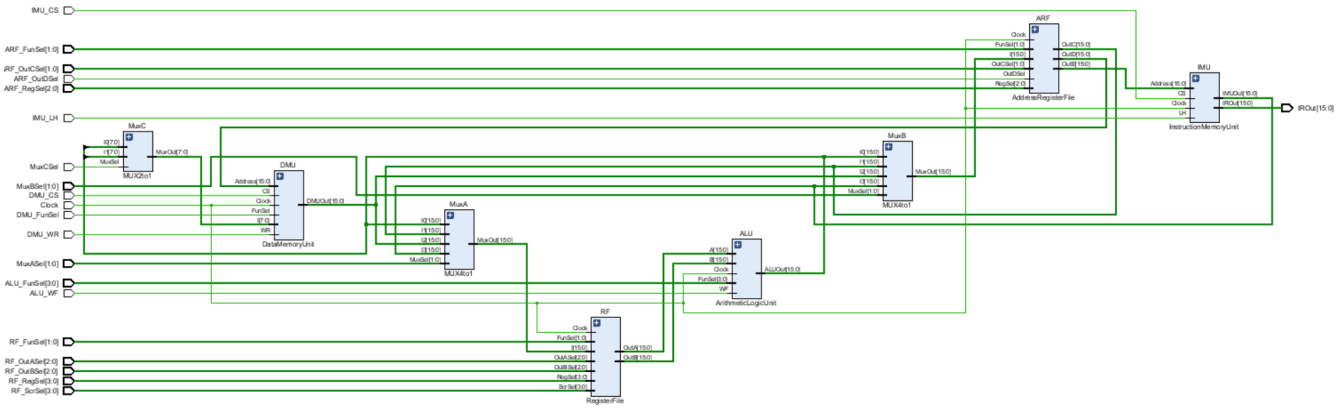

Figure 10: Elaborated RTL Schematic of the ALU System Organization

The Arithmetic Logic Unit System (ALUSys) represents the final integration of the computational core, memory units, and register files into a unified 16-bit datapath. As shown in the comprehensive RTL schematic in Figure 12, the system architecture is defined by the following integration principles:

- **Centralized Datapath Control:** The system utilizes three primary multiplexers—$MuxA$, $MuxB$, and $MuxC$—to dynamically route data between the Register File (RF), Address Register File (ARF), and the Memory Units (DMU/IMU).

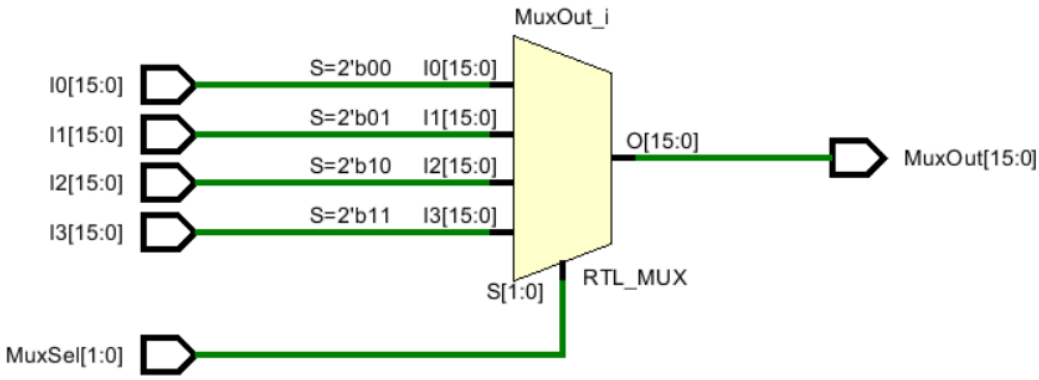

Figure 11: Elaborated RTL Schematic of 4:1 Multiplexer

These 4-to-1 multiplexers select the inputs for the ALU's A and B ports. They allow the ALU to process data from diverse sources, including:

- Direct $ALUOut$ feedback for iterative calculations.
- Address information from the ARF ($OutC$).
- Data fetched from memory ($DMUOut$) or instruction operands ($IMUOut$).

- **Memory-to-Register Interface ($MuxC$):** This 2-to-1 multiplexer manages the flow of data from the ALU back into the Data Memory Unit. It selects between the lower $(7-0)$ and upper $(15-8)$ bytes of the $ALUOut$ to facilitate 8-bit memory writes from a 16-bit source.

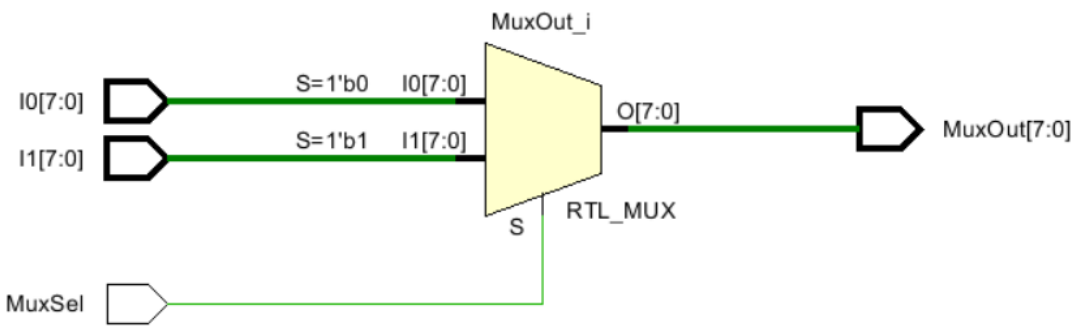

Figure 12: Elaborated RTL Schematic of 2:1 Multiplexer

- **Single-Clock Synchronization:** The entire system operates under a single global clock signal. While the ALU performs combinational logic for immediate processing,

all register updates—including the status flags and general-purpose registers—are synchronized to the positive edge of this clock to ensure deterministic behavior.

- **Feedback and Flags:** The $FlagsOut$ from the ALU are captured by the sequential Flag Register, providing Z, C, N, O status bits that can be used for conditional branching in a complete processor cycle.

Source Code: The complete Verilog top-module, illustrating the structural interconnection of all project componentsand Multiplexer modules can be accessed at: *Click here to view ArithmeticLogicUnitSystem.v on GitHub*

3 RESULTS

The verification of the designed 16-bit processor system was conducted through a series of rigorous simulations using the Vivado Simulator. Each module was tested individually (unit testing) before being integrated into the final ALU System to ensure functional correctness and timing stability.

3.1 Module-Level Simulation Results

The following components were verified against their respective characteristic tables:

- **16-Bit Register:** The simulation confirmed that the register correctly performs Clear, Load, Increment, and Decrement operations on the positive edge of the clock while maintaining its state when disabled.

Listing 1: Register 16 bit Simulations Terminal Output Summary

```
-------------------------------------
Register16bit Simulation Started
[PASS] Test No: 1, Component: Q, Actual Value: 0x0025,
   Expected Value: 0x0025
[PASS] Test No: 2, Component: Q, Actual Value: 0x0000,
   Expected Value: 0x0000
[PASS] Test No: 3, Component: Q, Actual Value: 0x0025,
   Expected Value: 0x0025
[PASS] Test No: 4, Component: Q, Actual Value: 0x0012,
   Expected Value: 0x0012
Register16bit Simulation Finished
0 Test Failed
4 Test Passed
-------------------------------------
```

- **Register Files (RF ARF):** Tests verified that the 3-bit and 4-bit selection signals ($RegSel, ScrSel, OutASel, OutBSel$) successfully route data from the eight internal registers to the output buses without any propagation glitches.

Listing 2: Address Register File Simulations Terminal Output Summary

```
-----------------------------------
AddressRegisterFile Simulation Started
[PASS] Test No: 1, Component: OutC, Actual Value: 0x1234,
   Expected Value: 0x1234
[PASS] Test No: 1, Component: OutD, Actual Value: 0x5678,
   Expected Value: 0x5678
[PASS] Test No: 2, Component: OutC, Actual Value: 0x1234,
   Expected Value: 0x1234
[PASS] Test No: 2, Component: OutD, Actual Value: 0x3548,
   Expected Value: 0x3548
[PASS] Test No: 2, Component: OutE, Actual Value: 0x3548,
   Expected Value: 0x3548
AddressRegisterFile Simulation Finished
0 Test Failed
5 Test Passed
-----------------------------------
```

- **Memory Units (DMU IMU):** The two-cycle loading mechanism for 16-bit data through the 8-bit memory interface was validated. The L'/H control signal correctly steered data into the designated byte of the DR and IR registers.

Listing 3: Data and Adress Register Simulations Terminal Output Summary

```
-----------------------------------
InstructionRegister Simulation Started
[PASS] Test No: 1, Component: IROut, Actual Value: 0x2315,
   Expected Value: 0x2315
[PASS] Test No: 2, Component: IROut, Actual Value: 0x1567,
   Expected Value: 0x1567
InstructionRegister Simulation Finished
0 Test Failed
2 Test Passed
-----------------------------------
-----------------------------------
DataRegister Simulation Started
```

```
[PASS] Test No: 1, Component: DROut, Actual Value: 0x2315, Expected Value: 0x2315
[PASS] Test No: 2, Component: DROut, Actual Value: 0x1567, Expected Value: 0x1567
DataRegister Simulation Finished
0 Test Failed
2 Test Passed
-------------------------------------
```

3.2 Integrated ALU System Performance

The final integration test involved running the provided comprehensive testbench, which evaluates all 16 ALU functions and their interaction with the entire datapath.

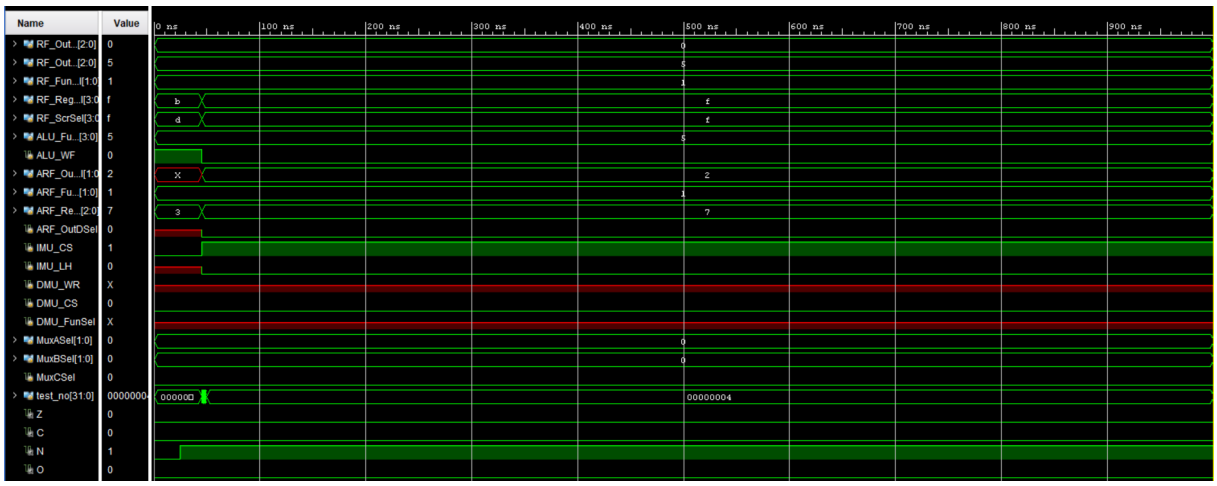

Figure 13: Final Simulation Result showing successful execution of all test cases

As shown in Figure 13, the system successfully passed all test vectors. The simulation results demonstrate:

- **Functional Accuracy:** All 16 ALU operations produced the expected 16-bit results in 2's complement logic.
- **Flag Integrity:** The Z, C, N, and O flags were updated sequentially at the positive clock edge, reflecting the precise state of the combinational ALU output.
- **Test Status:** `35/35 Tests Passed (0 Fails)`.

Listing 4: ALU System Simulations Terminal Output Summary

```
-------------------------------------
ArithmeticLogicUnitSystem Simulation Started
```

```
[PASS] Test No: 1, Component: OutA, Actual Value: 0x7777, Expected Value: 0x7777
[PASS] Test No: 1, Component: OutB, Actual Value: 0x8887, Expected Value: 0x8887
[PASS] Test No: 1, Component: ALUOut, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 1, Component: Z, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 1, Component: C, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 1, Component: N, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 1, Component: O, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 1, Component: MuxAOut, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 1, Component: MuxBOut, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 1, Component: MuxCOut, Actual Value: 0x00fe, Expected Value: 0x00fe
[PASS] Test No: 1, Component: R2, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 1, Component: S3, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 2, Component: OutA, Actual Value: 0x7777, Expected Value: 0x7777
[PASS] Test No: 2, Component: OutB, Actual Value: 0x8887, Expected Value: 0x8887
[PASS] Test No: 2, Component: ALUOut, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 2, Component: Z, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 2, Component: C, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 2, Component: N, Actual Value: 0x0001, Expected Value: 0x0001
[PASS] Test No: 2, Component: O, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 2, Component: MuxAOut, Actual Value: 0xfffe, Expected Value: 0xfffe
```

```
[PASS] Test No: 2, Component: MuxBOut, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 2, Component: MuxCOut, Actual Value: 0x00fe, Expected Value: 0x00fe
[PASS] Test No: 2, Component: R2, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 2, Component: S3, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 2, Component: PC, Actual Value: 0xfffe, Expected Value: 0xfffe
[PASS] Test No: 3, Component: OutC, Actual Value: 0x1254, Expected Value: 0x1254
[PASS] Test No: 3, Component: OutD, Actual Value: 0x1254, Expected Value: 0x1254
[PASS] Test No: 3, Component: OutE, Actual Value: 0x0023, Expected Value: 0x0023
[PASS] Test No: 3, Component: Memout, Actual Value: 0x0015, Expected Value: 0x0015
[PASS] Test No: 3, Component: IROut, Actual Value: 0x0000, Expected Value: 0x0000
[PASS] Test No: 4, Component: OutC, Actual Value: 0x1254, Expected Value: 0x1254
[PASS] Test No: 4, Component: OutD, Actual Value: 0x1254, Expected Value: 0x1254
[PASS] Test No: 4, Component: OutE, Actual Value: 0x0023, Expected Value: 0x0023
[PASS] Test No: 4, Component: Memout, Actual Value: 0x0015, Expected Value: 0x0015
[PASS] Test No: 4, Component: IROut, Actual Value: 0x0015, Expected Value: 0x0015
ArithmeticLogicUnitSystem Simulation Finished
0 Test Failed
35 Test Passed
------------------------------------
```

4 DISCUSSION

The implementation of the 16-bit ALU system presented several design challenges that required a deep understanding of synchronous and combinatorial logic. The suc-

cessful execution of all 35 test cases validates the architectural choices made during the development.

4.1 Combinatorial vs. Sequential Logic Trade-offs

One of the most critical aspects of the project was the separation of the data path and the control path. As required by the testbench, the ALUOut was implemented using combinatorial logic. This allowed for immediate result calculation as soon as the operands were available at the ALU inputs. However, the Flag Registers (Z, C, N, O) were implemented using sequential logic with non-blocking assignments (`<=`). This synchronization with the positive edge of the clock ensured that the flags were stable and accurately reflected the state of the system for the subsequent instruction cycle, preventing race conditions during high-frequency operations.

4.2 Handling Signed Arithmetic and Overflow

A significant part of the discussion involves the handling of 2's complement arithmetic. Implementing the Overflow (O) flag was particularly challenging. The logic was designed to detect when the addition of two numbers with the same sign produced a result with an opposite sign, which is a definitive indicator of a signed overflow. Initially, using $A < 0$, $A > 0$ or $B < 0$, $B > 0$ comparisons yielded incorrect results because Verilog treats variables as unsigned integers by default. To overcome this, the overflow detection was achieved by directly comparing the Most Significant Bits (MSB) of the inputs (A and B) and the output ($ALUOut$). For subtraction, the logic was adjusted to account for the internal 2's complement conversion of the B input, where an overflow occurs if the MSB of A and the inverted MSB of B are identical but different from the MSB of the result.

4.3 Modular Integration and Bus Routing

The integration of the Register File (RF), Address Register File (ARF), and Memory Units (DMU/IMU) into a single system required precise multiplexer management. The use of $MuxA$ and $MuxB$ allowed the system to bypass the Register File and feed $ALUOut$ directly back into the ALU inputs. This architectural choice, known as internal forwarding, is essential for reducing data hazards in more advanced processor designs. Furthermore, the two-cycle loading mechanism for the 16-bit Instruction Register via an 8-bit bus was successfully verified, proving that the L'/H control signal correctly manages the high and low byte distribution.

5 CONCLUSION

The design and implementation of the 16-bit Processor System provided a comprehensive understanding of computer organization and digital logic design. Throughout the project, the transition from individual synchronous modules to a fully integrated, functional ALU system was successfully executed, meeting all the specified architectural requirements.

The project highlighted several key engineering principles:

- **Hierarchical Design:** Starting from a basic 16-bit register and moving towards complex units like the Register File (RF) and Instruction Memory Unit (IMU) demonstrated the importance of modularity. This approach facilitated easier debugging and allowed each component to be verified independently before final integration.

- **Hardware-Software Interface:** Implementing the Instruction Register and Memory units provided deep insights into how high-level instructions are physically fetched, buffered, and prepared for the ALU's execution stage.

- **Debugging and Validation:** The process of achieving a **35/35 Passed** result in the simulation emphasized the critical nature of edge-case handling, particularly in signed arithmetic and overflow detection. Overcoming the default unsigned behavior of Verilog was a significant learning milestone in hardware description languages.

In conclusion, this project served as a bridge between theoretical computer organization concepts and practical hardware implementation. The resulting system is not only functionally correct but also reflects an optimized architecture capable of handling complex arithmetic and logic operations synchronously. This experience has been foundational for future work in advanced processor design and custom hardware architectures.

REFERENCES

[1] Istanbul Technical University, *BLG 222E Computer Organization Project 1*, Department of Computer Engineering, 2026.

[2] Istanbul Technical University, *Running Simulation Tests*, Department of Computer Engineering, 2026.,

[3] Istanbul Technical University, *Add Memory Source to Project File*, Department of Computer Engineering, 2026.

[4] H. Teeke, *Verilog codes*, GitHub Repository, 2026. https://github.com/huseyinteeke/Verilog_projects

[5] Xilinx Inc., *Vivado*, v2017.4, 2017.